



Instituto de Computação - IC
PPGI016-11 - TÓPICOS ESPECIAIS EM ENGENHARIA DE SISTEMAS
COMPUTACIONAIS: NAVEGAÇÃO DE ROBÔS

Navegação de Robôs baseada em campos potenciais

João Arthur Gaia da Rocha Almeida

Professor: Glauber Rodrigues Leite

Maceió - AL
16/06/2026

1 Introdução

A navegação de robôs é a base da robótica móvel. E extrapola conceitos como a cinemática, pois precisa levar em consideração o ambiente, que pode conter obstáculos móveis e fixos, além de espaços não euclidianos, como o oceano por exemplo, e a presença de fluxos que influenciam no robô, como vento e correnteza. Dessa maneira, em cenários onde o ambiente é conhecido, destacam-se os campos potenciais.

2 Funcionamento do Algoritmo

O planejamento de rota baseado em campos potenciais consiste em criar funções atrativas e repulsivas, para fazer com que o robô se mova em direção a um objetivo e desvie dos obstáculos, seguindo a direção do gradiente descendente do resultado dessas funções [1]. Nessa abordagem, um mapa 2D com obstáculos, será representado matematicamente como uma série de vales e colinas. O robô vai se moverem direção ao vale mais fundo e evitar subir as colinas. O perigo mora em parar em um vale e ele não ser o mais baixo, isto é, "cair" em um mínimo local.

A função atrativa é comumente definida por função cônica (I), que consiste na distância (d) entre o ponto do robô (q) e o ponto de objetivo (q_{goal}), multiplicado por um ganho atrativo (ζ).

$$U(q) = \zeta d(q, q_{goal}) \quad (\text{I})$$

Ao computar o gradiente atrativo de I, chegamos a II, que possui duas questões, a indefinição quando $q = q_{goal}$ e uma descontinuidade na origem quando esse algoritmo é representado numericamente.

$$\nabla U(q) = \frac{\zeta}{d(q, q_{goal})} (q - q_{goal}) \quad (\text{II})$$

Para evitar esses problemas, recorre-se a uma função parabólica (III), que considera o quadrado da distância entre q e q_{goal} .

$$U_{att}(q) = \frac{1}{2} \zeta d^2(q, q_{goal}) \quad (\text{III})$$

Que, por sua vez, é continuamente diferenciável e chega-se a IV

$$\nabla U_{att}(q) = \zeta d(q, q_{goal}) \quad (\text{IV})$$

Porém, por se tratar III é quadrática, quando q está longe de q_{goal} , a magnitude do gradiente se torna muito grande, o que faz com que o robô se mova muito rápido. Com uma velocidade acima do aceitável, para contornar isso, se utiliza uma combinação das funções cônica e parabólica (VI), onde d_{goal}^* é o limiar da distância que faz a troca entre as funções quadrática e cônica.

$$U_{att}(q) = \begin{cases} \frac{1}{2} \zeta d^2(q, q_{goal}), & d(q, q_{goal}) \leq d_{goal}^*, \\ d_{goal}^* \zeta d(q, q_{goal}) - \frac{1}{2} \zeta (d_{goal}^*)^2, & d(q, q_{goal}) > d_{goal}^* \end{cases} \quad (\text{VI})$$

Que, por sua vez, tem como gradiente a expressão VII.

$$\nabla U_{att}(q) = \begin{cases} \zeta (q - q_{goal}), & d(q, q_{goal}) \leq d_{goal}^*, \\ \frac{d_{goal}^* \zeta (q - q_{goal})}{d(q, q_{goal})}, & d(q, q_{goal}) > d_{goal}^* \end{cases} \quad (\text{VII})$$

Dessa forma, o robô se move rápido quando está longe do alvo e mais lentamente próximo dele.

Por outro lado, as funções repulsivas tem dois casos básicos: O obstáculo está muito longe para influir, ou está suficientemente perto para influir. Dessa maneira, para o primeiro caso, a força repulsiva é nula e o robô só será movido pela força atrativa. Caso contrário, o robô deverá se afastar do obstáculo, isto é feito pela função VIII

$$U_{rep}(q) = \begin{cases} \frac{1}{2} \eta \left(\frac{1}{D(q)} - \frac{1}{Q^*} \right)^2, & D(q) \leq Q^*, \\ 0, & D(q) > Q^* \end{cases} \quad (\text{VIII})$$

Onde Q^* é o limiar que define se um obstáculo está perto o suficiente para ser ou não ignorado pelo robô e $D(q)$ é a distância para o obstáculo mais próximo do robô e η é o ganho repulsivo, o quanto o robô deve se afastar do obstáculo. Com isso, calcula-se o gradiente dessa função (IX).

$$\nabla U_{rep}(q) = \begin{cases} \frac{1}{2}\eta(\frac{1}{Q^*} - \frac{1}{D(q)})^2 \frac{1}{D^2(q)} \nabla D(q), & D(q) \leq Q^*, \\ 0, & D(q) > Q^* \end{cases} \quad (IX)$$

Com essas funções (VI e VIII) e seus gradientes (VII e IX), têm-se a direção que o robô deverá seguir para chegar no objetivo.

3 Cenário de Simulação

Uma simulação foi feita no CoppeliaSimEDU, o robô foi simulado por uma esfera de diâmetro 0,2m, os obstáculos foram representados por cubos de aresta 0,2m, o objetivo é representado pelo círculo vermelho, como visto em 1.

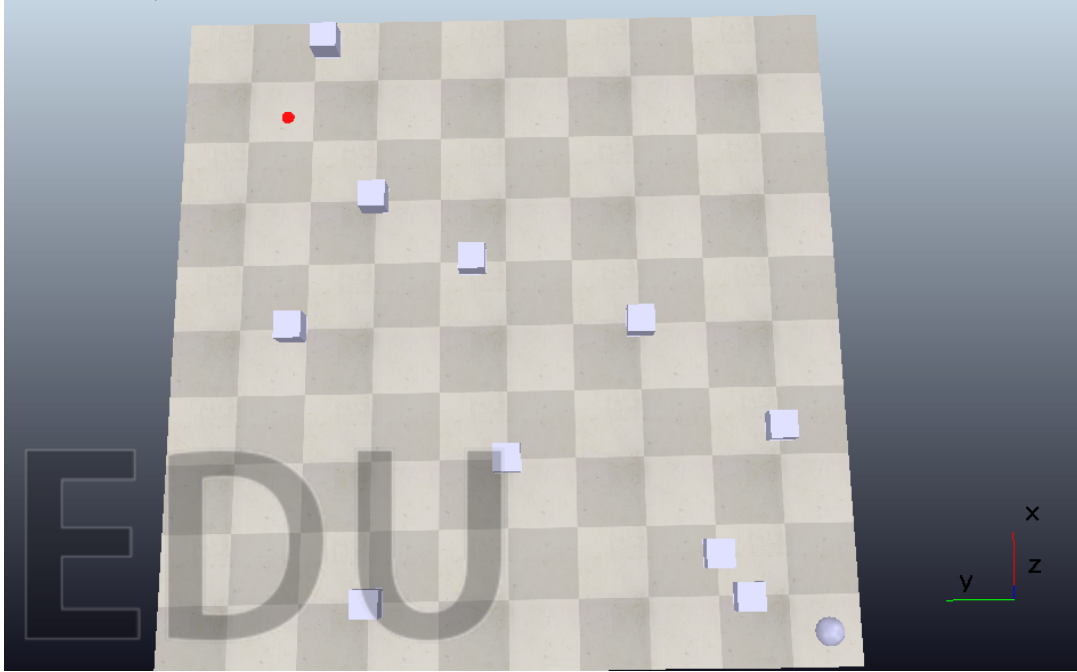


Figura 1: Mapa da simulação

As entidades estão posicionadas de acordo com a tabela 1. Um segundo cenário foi feito nas mesmas condições, com a diferença do obstáculo 8 estar na posição (-1, -2).

Entidade	X	Y
Início	-2.250	-2.250
Objetivo	1.7	1.7
Obstáculo 0	-2	-1.7
Obstáculo 1	-1	0
Obstáculo 2	0.5	0.25
Obstáculo 3	1	1
Obstáculo 4	2.3	1.4
Obstáculo 5	-1.7	-1.5
Obstáculo 6	0	-1
Obstáculo 7	-2	1
Obstáculo 8	-0.8	-2
Obstáculo 9	0	1.6

Tabela 1: Posições da simulação

O algoritmo foi implementado em um script em python, disponível do [github da atividade](#), com os parâmetros, em ambos os cenários, de acordo com a tabela 2. Onde dt é o passo de integração da simulação.

Parâmetro	Valor
ζ	2.0
d^*	0.25
η	10.0
Q^*	0.5
dt	0.01

Tabela 2: Parâmetros da simulação

4 Resultados

Para o primeiro cenário, o robô se move como o esperado, desviando dos obstáculos e chegando no objetivo, como pode ser visto na figura 2.

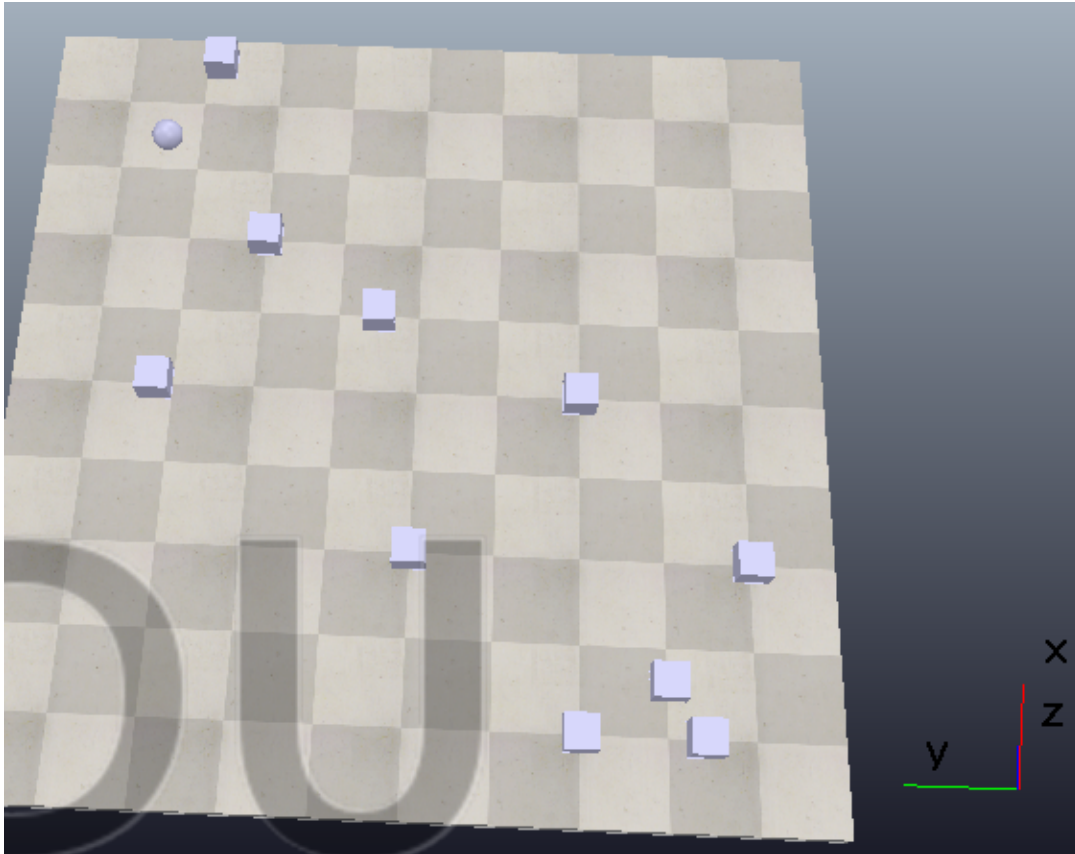


Figura 2: Primeiro cenário

Para o segundo cenário, o robô "trava" em um mínimo local nos obstáculos, sem conseguir seguir chegar ao objetivo, como visto na figura 3.

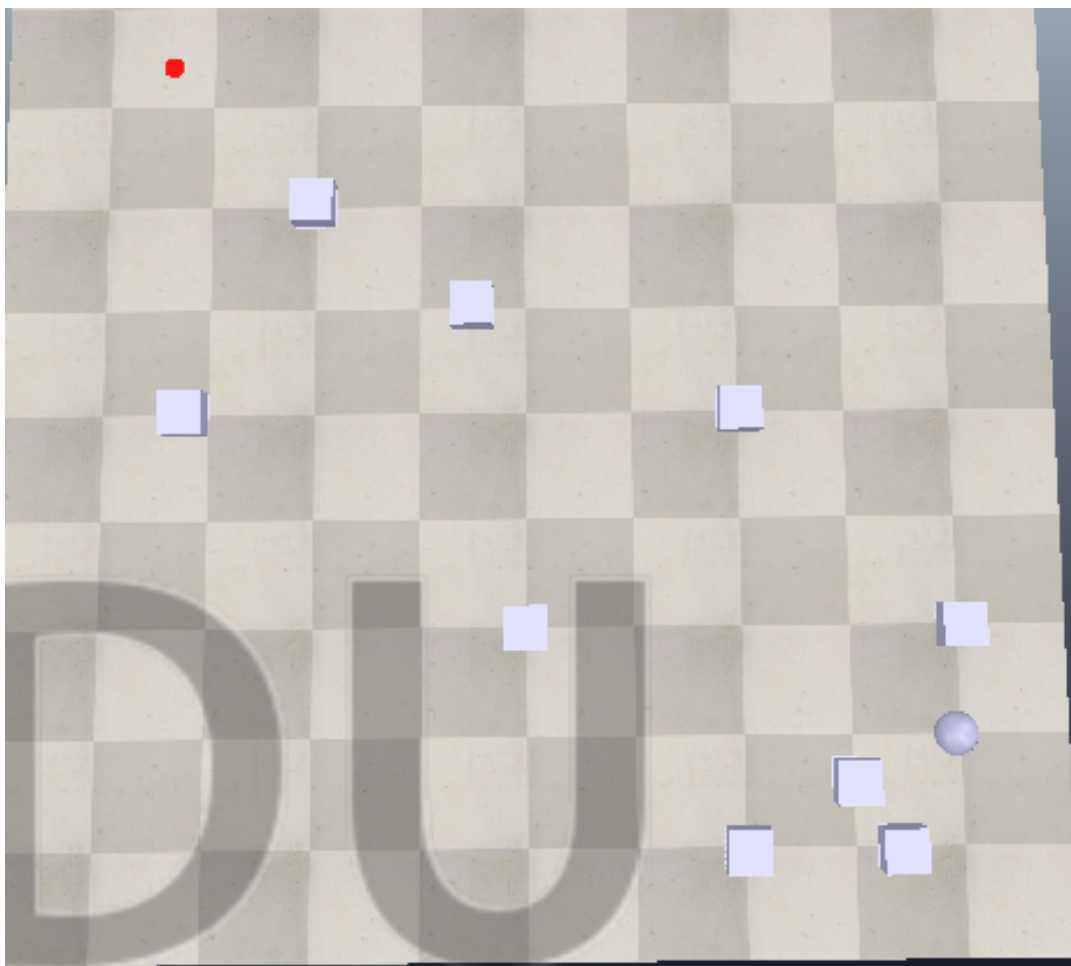


Figura 3: Segundo cenário

Ambos os experimentos e a cena do CoppeliaSim podem ser vistos nos vídeos no [github da atividade](#).

5 Conclusão

O algoritmo de campos potenciais é fácil de implementar e de entender, em geral funciona bem, porém há a grande limitação de estar sujeito a mínimos locais, como exposto pelo segundo cenário.

Uma possibilidade futura é ajustar os parâmetros de tal maneira que o robô consiga ultrapassar esse mínimo local e chegar ao objetivo. Porém, esse fine-tuning sempre será limitado a uma disposição específica de obstáculos. Para contornar isso, somente implementando algoritmos mais complexos, como o Wave Front, por exemplo.

Referências

- [1] H. M. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun. *Principles of robot motion*. Intelligent Robotics and Autonomous Agents series. Bradford Books, Cambridge, MA, May 2005.